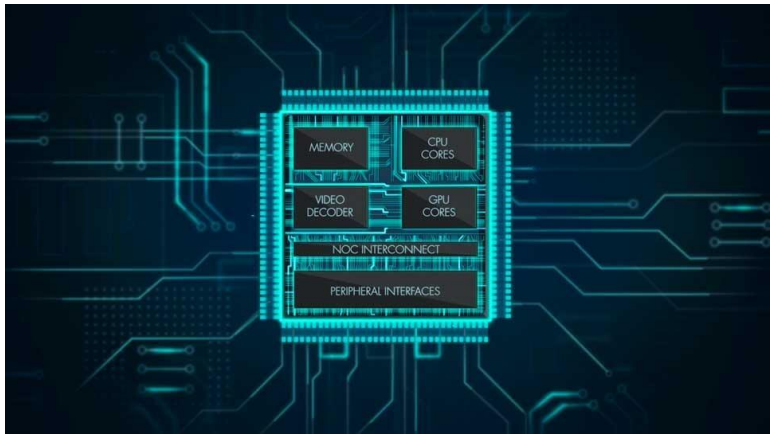
Digital System Design

Basic Practice 4



Date : 2026. 04. 16.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

Contents

- Data Reuse
- Problem 1: Single Channel Conv1d (w/ 3 PEs)
- Problem 2: Multi Channel Conv1d (w/ 9 PEs)

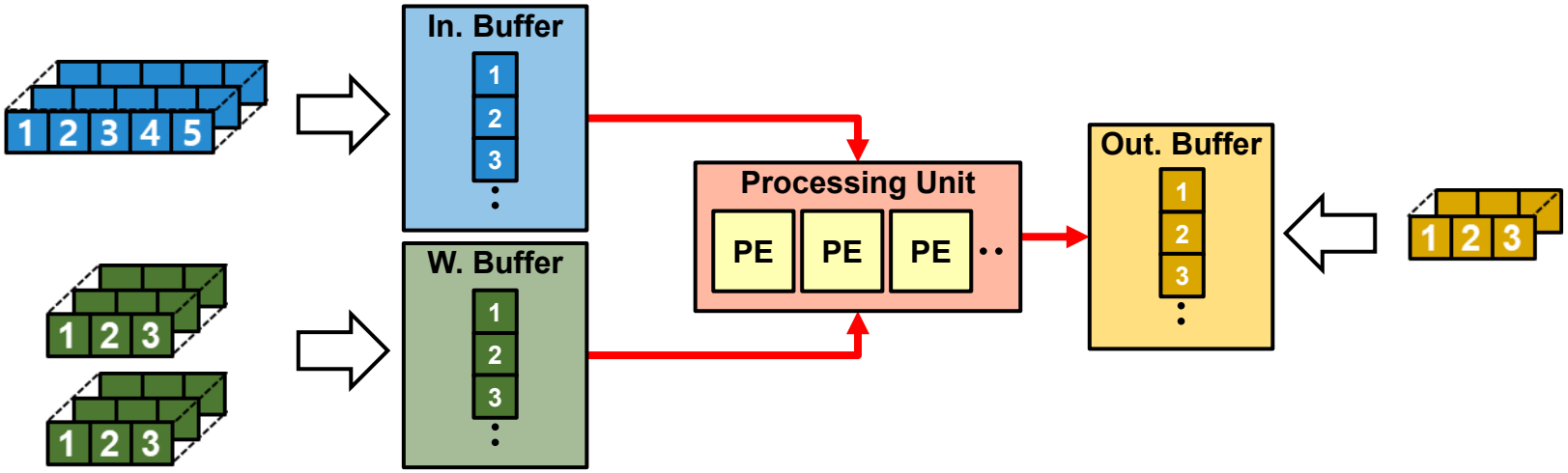
Contents

- Data Reuse
- Problem 1: Single Channel Conv1d (w/ 3 PEs)
- Problem 2: Multi Channel Conv1d (w/ 9 PEs)

Limitations of Hardware Mapping

Hardware cannot directly represent multi-dimensional data structures

- In Hardware, dimensions are flattened into **linear data streams.**
- As a result, the **computation structure of Conv1d** cannot be directly mapped onto hardware.
- Therefore, computation must be **restructured into a hardware-friendly dataflow.**
- The dataflow is designed to support, **sequential data movement, data reuse, parallel execution.**

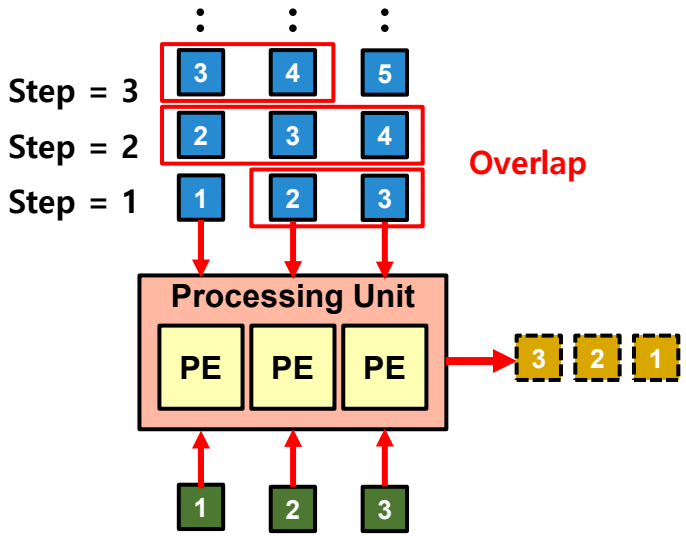


< From Multi-dimensional Computation to Hardware Dataflow >

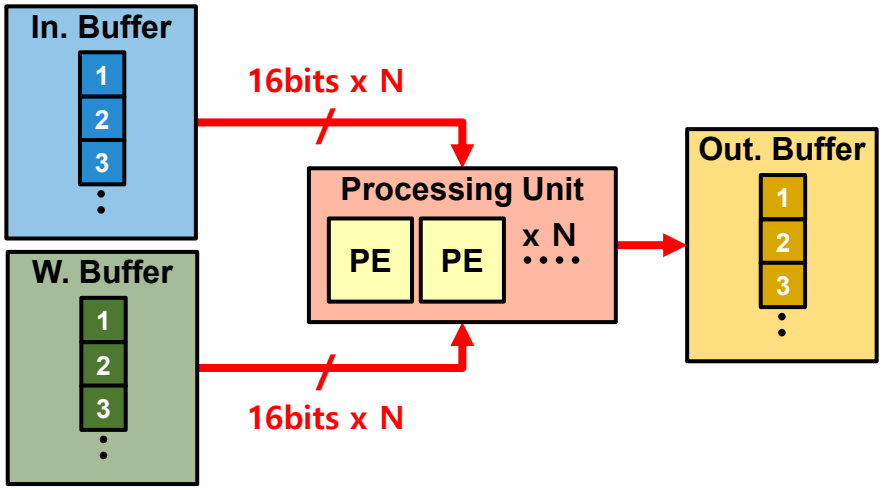
Conv1d Data Access Pattern

Conv1d follows a sliding-window-based data pattern.

- The sliding window introduces **overlap between input data.**
- This causes **redundant data movement,** as the same data is transferred multiple times.
- As parallelism increases**, more data is required simultaneously**, resulting in **higher memory access demand.**
- However, **memory bandwidth** is limited and **cannot fully support this demand.**
- Consequently, memory bandwidth becomes the primary bottleneck.



< Overlapping Data in Conv1d >

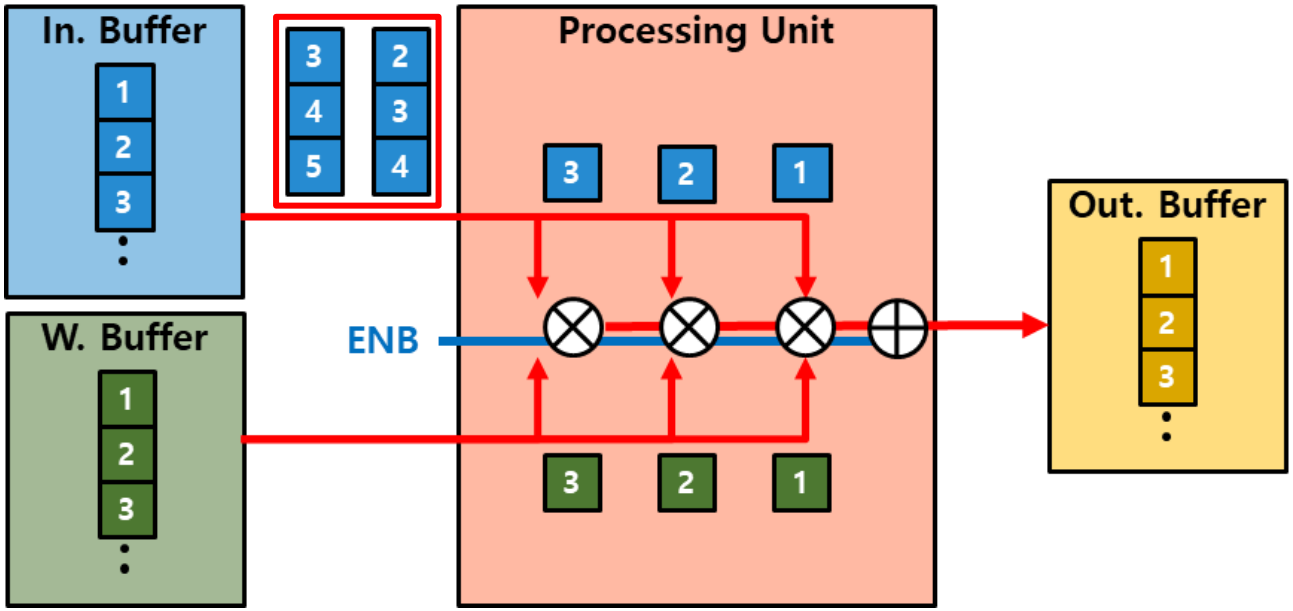


< Memory bandwidth limits scalability of parallel PEs >

Improving Data Reuse in Hardware (1)

Naive Streaming (Redundant Memory Access)

- Data is not **stored inside the PE** and is loaded from memory every cycle.
- The same data is repeatedly fetched from memory.
- Computation is **heavily dependent on memory access.**
- Memory bandwidth becomes the performance bottleneck.

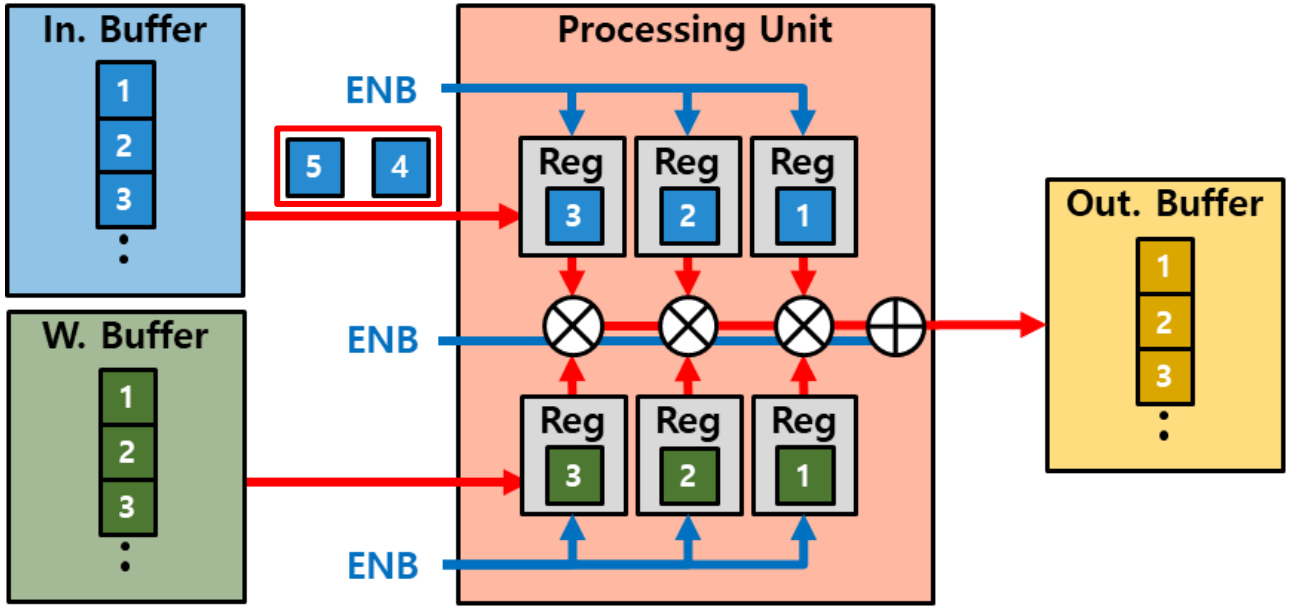


< Inefficient Architecture (No Data Reuse) >

Improving Data Reuse in Hardware (2)

Register-based Sliding Window (Efficient Data Reuse)

- Each PE locally holds activation and weight in its registers.
- The same data can be directly **reused from local register.**
- Streaming supplies only new data (existing data is retained).
- **Reduced memory access** and **improved PE utilization.**

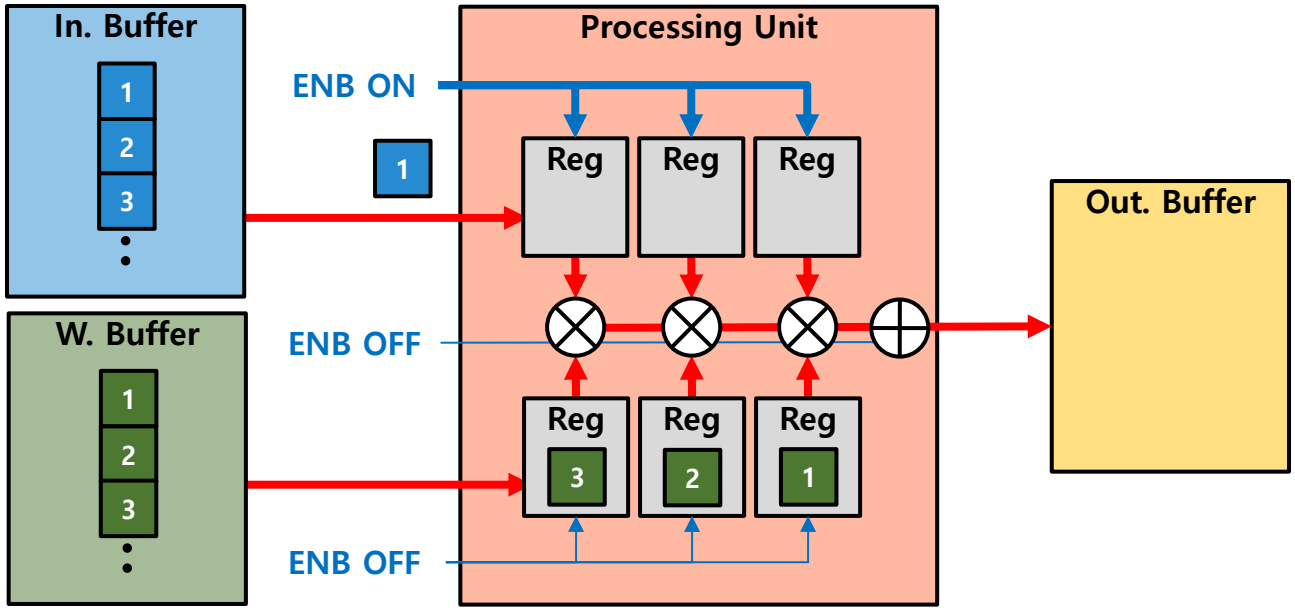


< Efficient Architecture (Data Reuse) >

Conv1d Execution Flow (1)

Example Conv1d Register-based Processing (w/ 3 PEs)

- Reuses data from local registers.

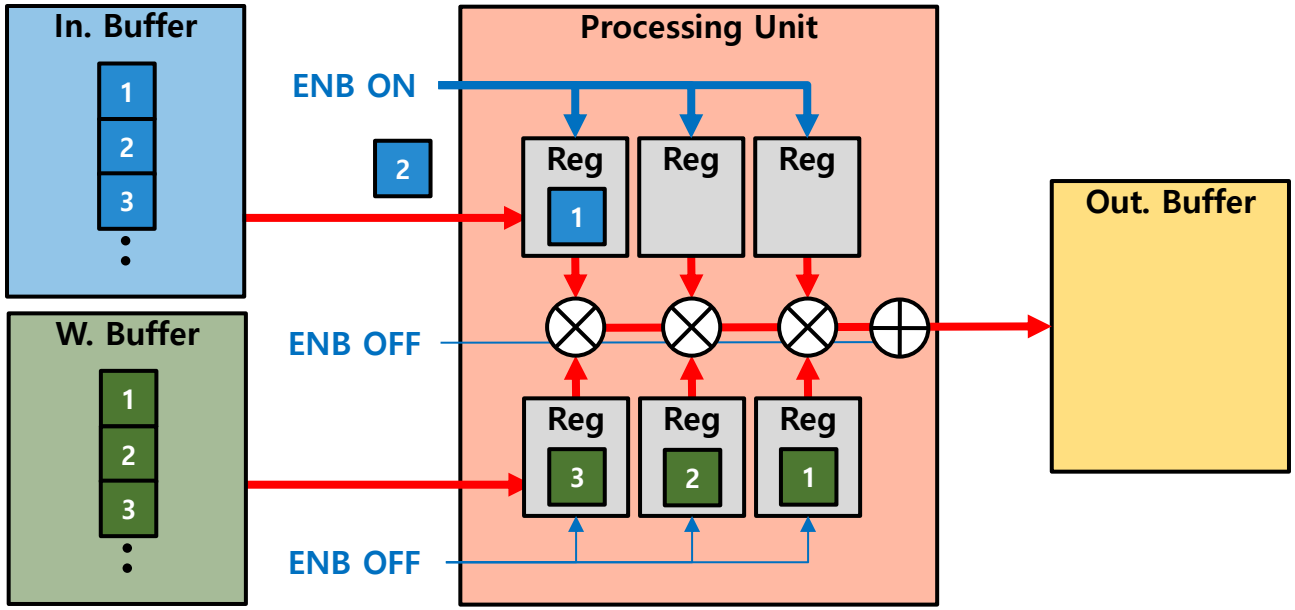


< Register-based Conv1d Execution (3 Pes) >

Conv1d Execution Flow (2)

Example Conv1d Register-based Processing (w/ 3 PEs)

- Reuses data from local registers.

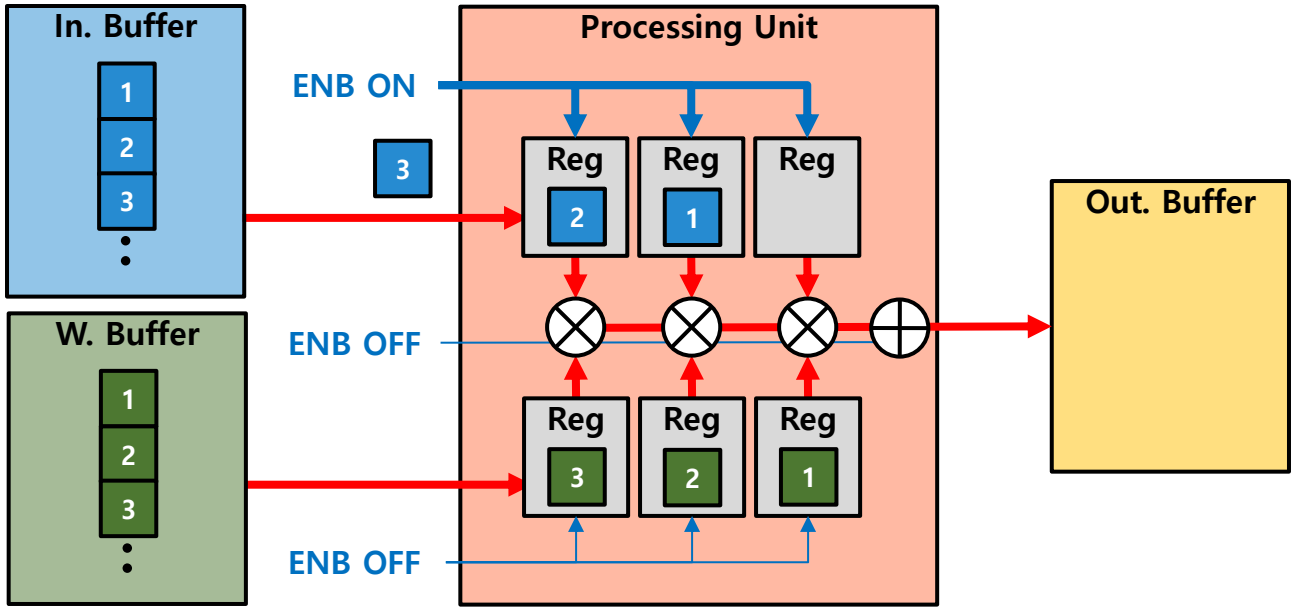


< Register-based Conv1d Execution (3 Pes) >

Conv1d Execution Flow (3)

Example Conv1d Register-based Processing (w/ 3 PEs)

- Reuses data from local registers.

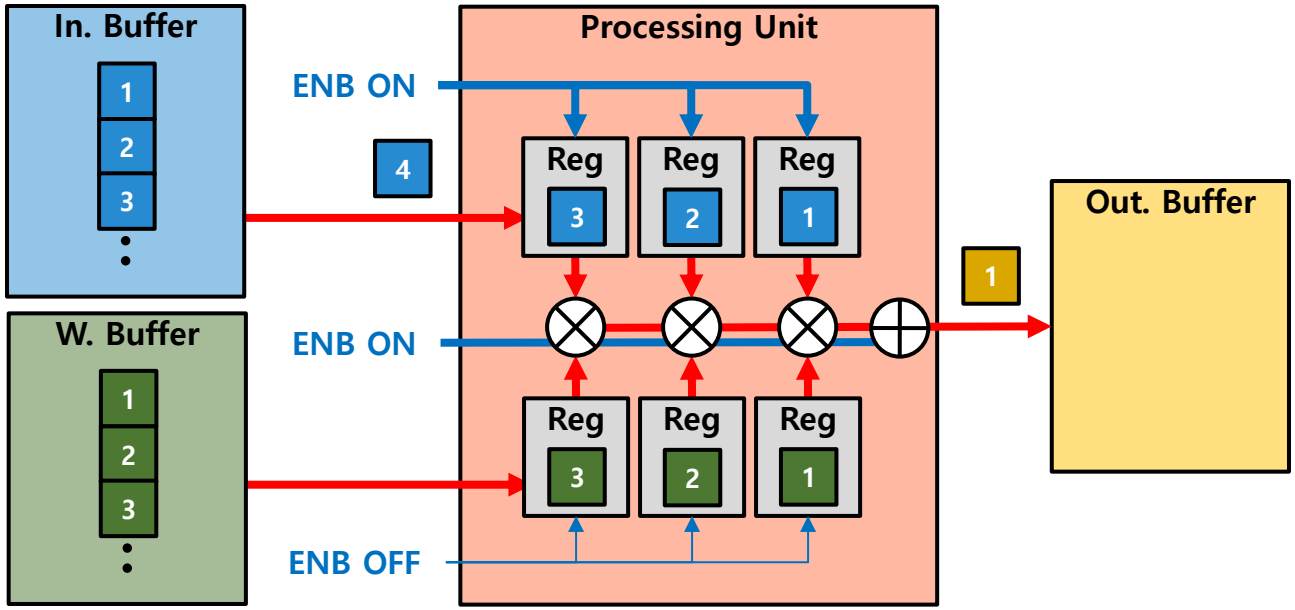


< Register-based Conv1d Execution (3 Pes) >

Conv1d Execution Flow (4)

Example Conv1d Register-based Processing (w/ 3 PEs)

- Reuses data from local registers.

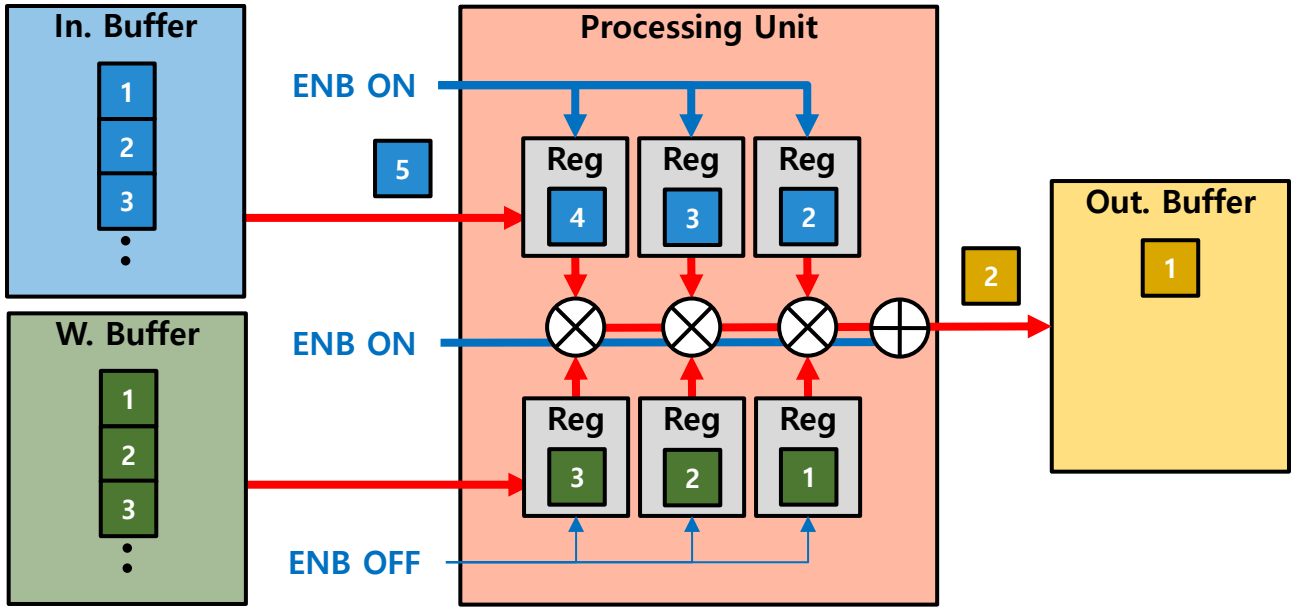


< Register-based Conv1d Execution (3 Pes) >

Conv1d Execution Flow (5)

Example Conv1d Register-based Processing (w/ 3 PEs)

- Reuses data from local registers.

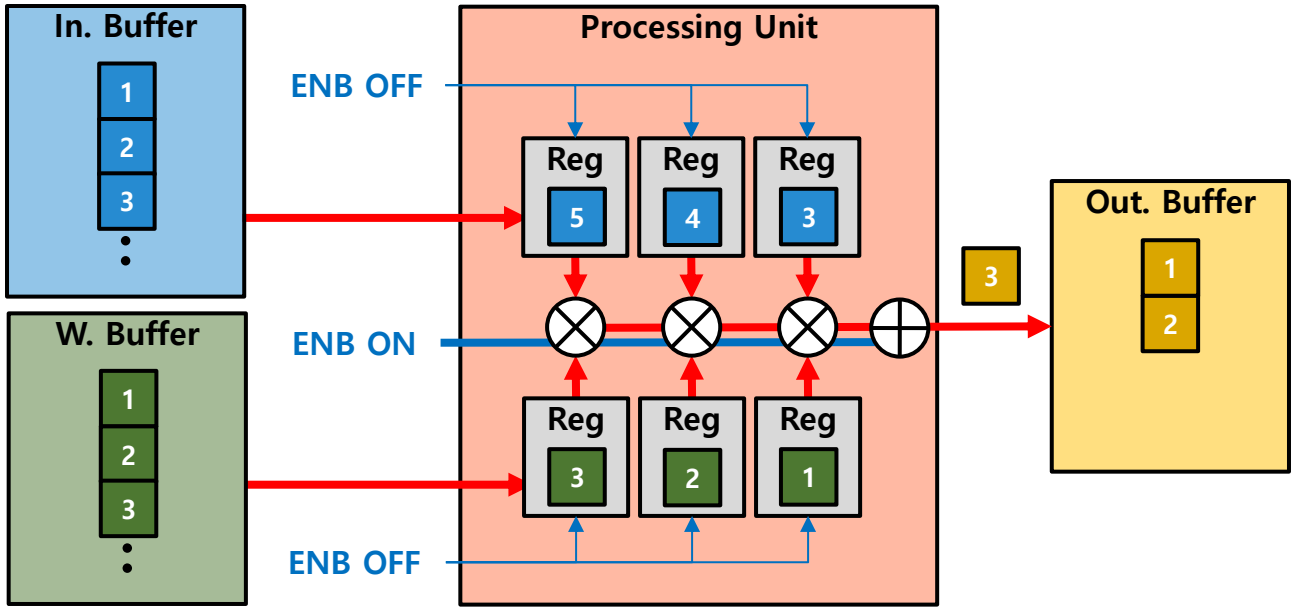


< Register-based Conv1d Execution (3 Pes) >

Conv1d Execution Flow (6)

Example Conv1d Register-based Processing (w/ 3 PEs)

- Reuses data from local registers.

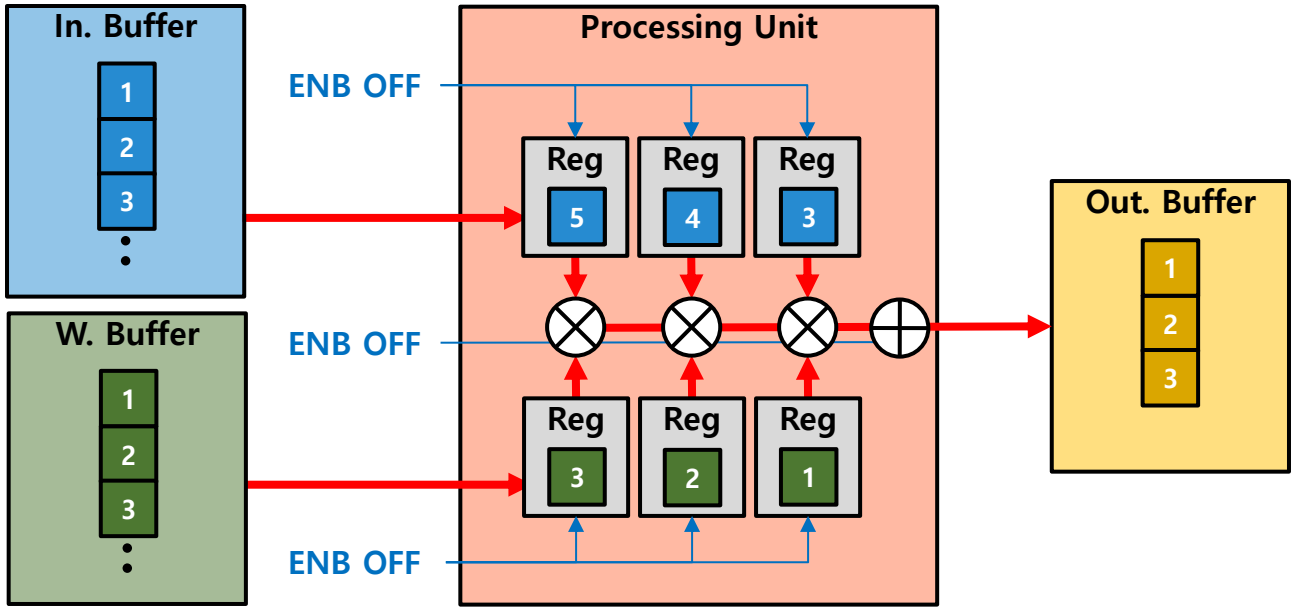


< Register-based Conv1d Execution (3 Pes) >

Conv1d Execution Flow (7)

Example Conv1d Register-based Processing (w/ 3 PEs)

- Reuses data from local registers.



< Register-based Conv1d Execution (3 Pes) >

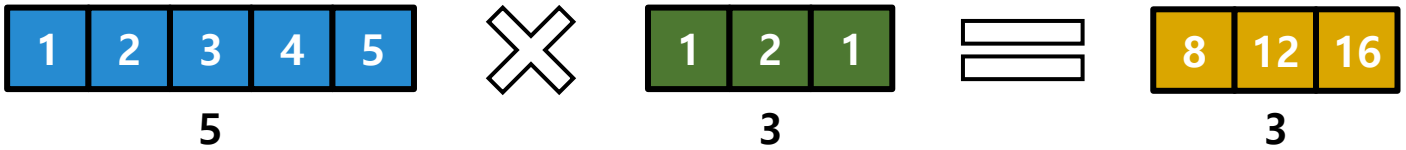
Contents

- Data Reuse
- Problem 1: Single Channel Conv1d (w/ 3 PEs)
- Problem 2: Multi Channel Conv1d (w/ 9 PEs)

Problem 1. Conv1d Design Objective

Single-Channel Input Conv1d Operation Objective

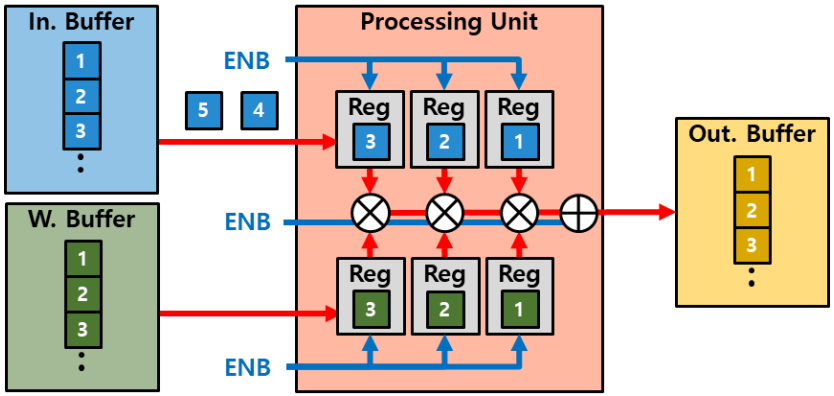
- Implement Conv1d operation to generate a **1×3 output from a 1×5 input and a 1×3 kernel.**
- Perform **sliding-window-based dot product (MAC)** computation at each position.



< Target Conv1d Operation >

Hardware Design Objective

- Store and reuse **activation and weight in registers.**
- Control data movement (shift) and computation using the **ENB signal.**
- **Accumulate MAC results** to generate the final output.

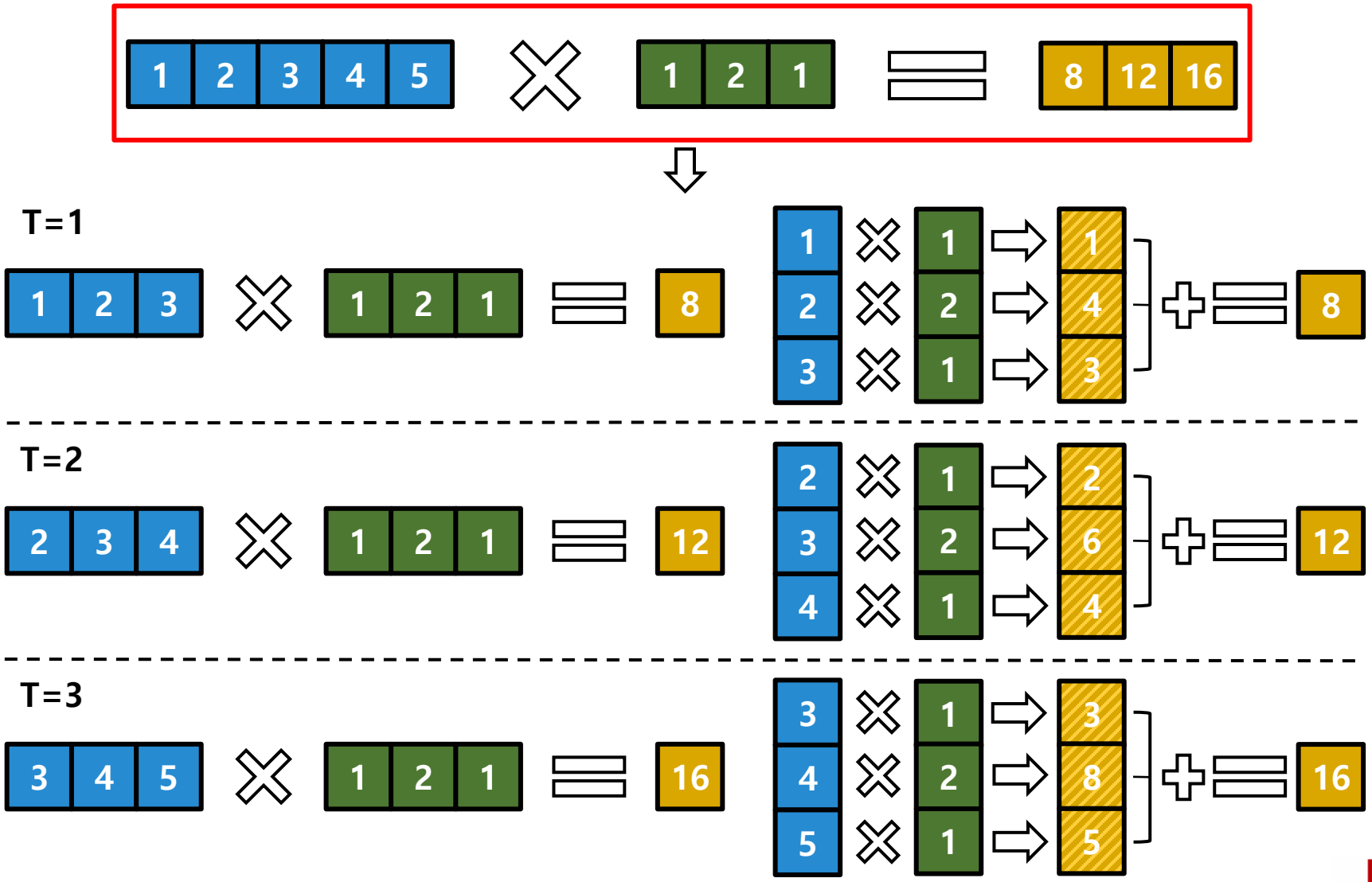


< Target Hardware Architecture >

IDSL
Intelligent Digital Systems Design Lab.
SEOULTECH

Detailed Computation Flow

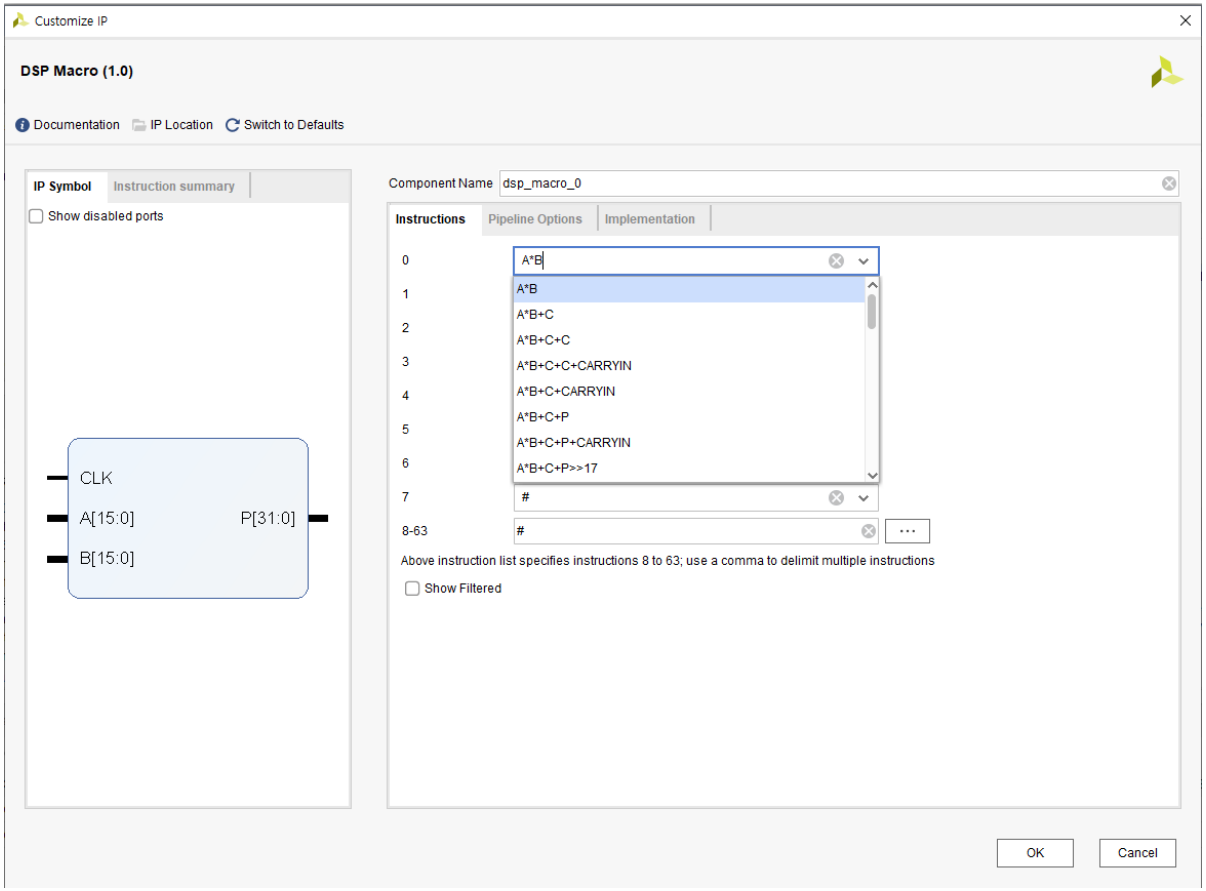
Target Conv1d: Detailed Computation Flow



DSP Generation – Remind (1)

IP Catalog – DSP Macro Configuration

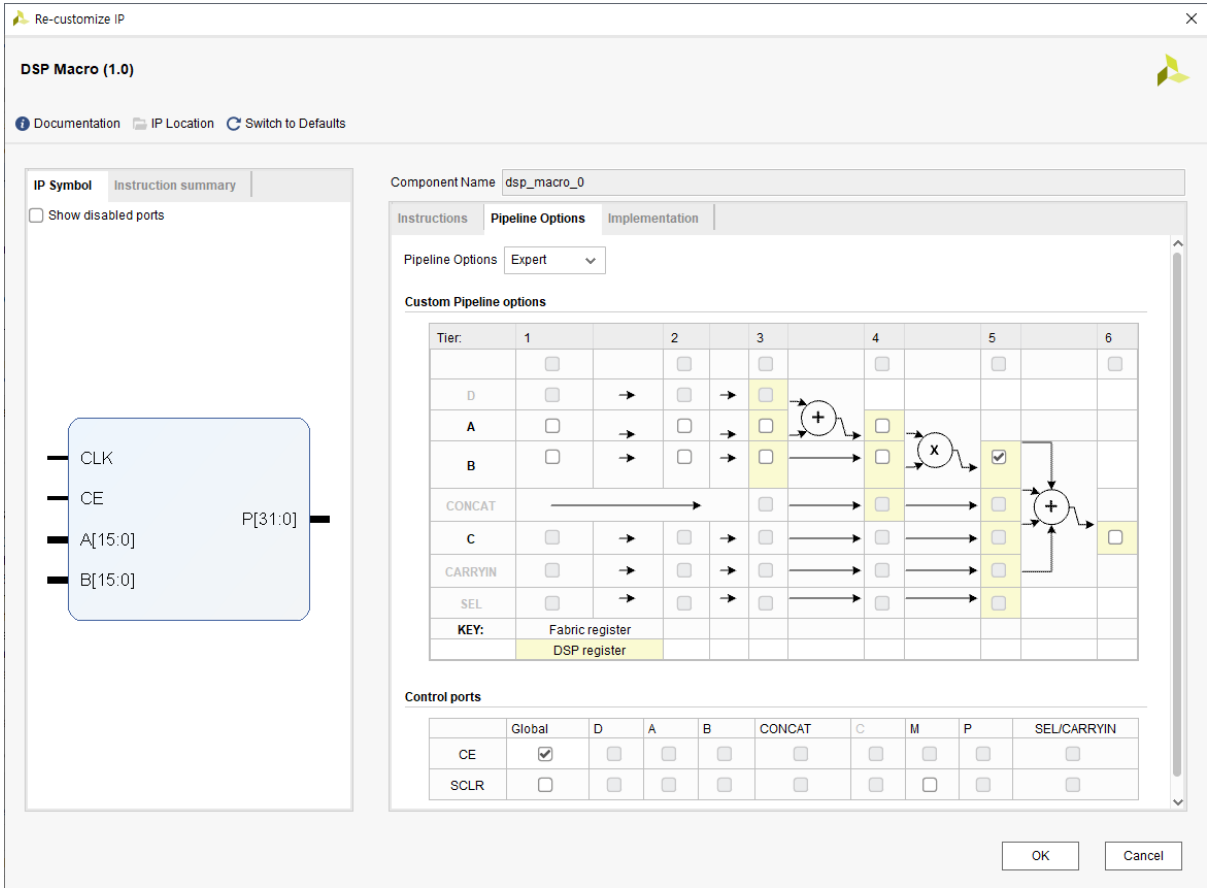
- Instruction: **A × B** (Multiplier mode).



DSP Generation – Remind (2)

IP Catalog – DSP Macro Configuration

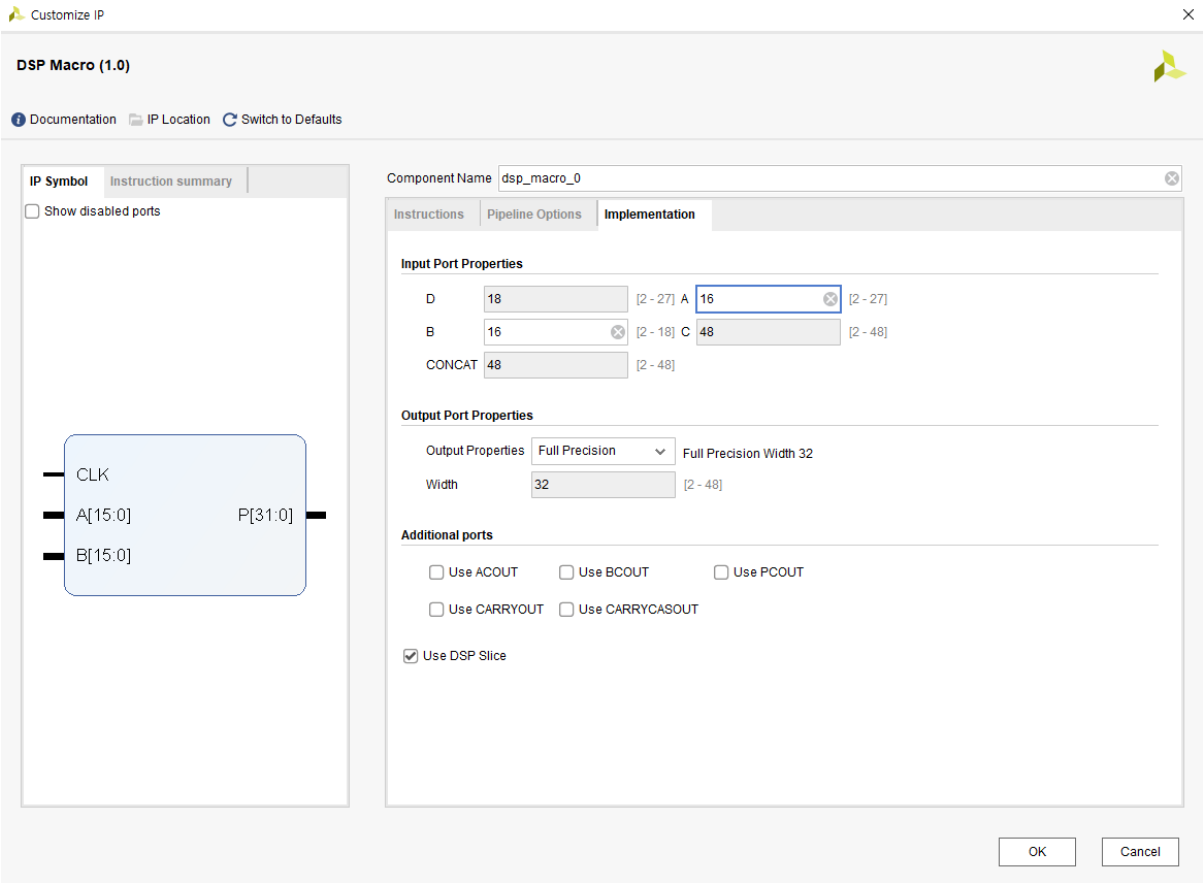
- **CE (Clock Enable) enabled** for controlled operation.



DSP Generation – Remind (3)

IP Catalog – DSP Macro Configuration

- Input Width: **A = 16 bit, B = 16 bit.**



BRAM Generation - Remind

Problem 1

Simple Dual-Port BRAM Inferring Code

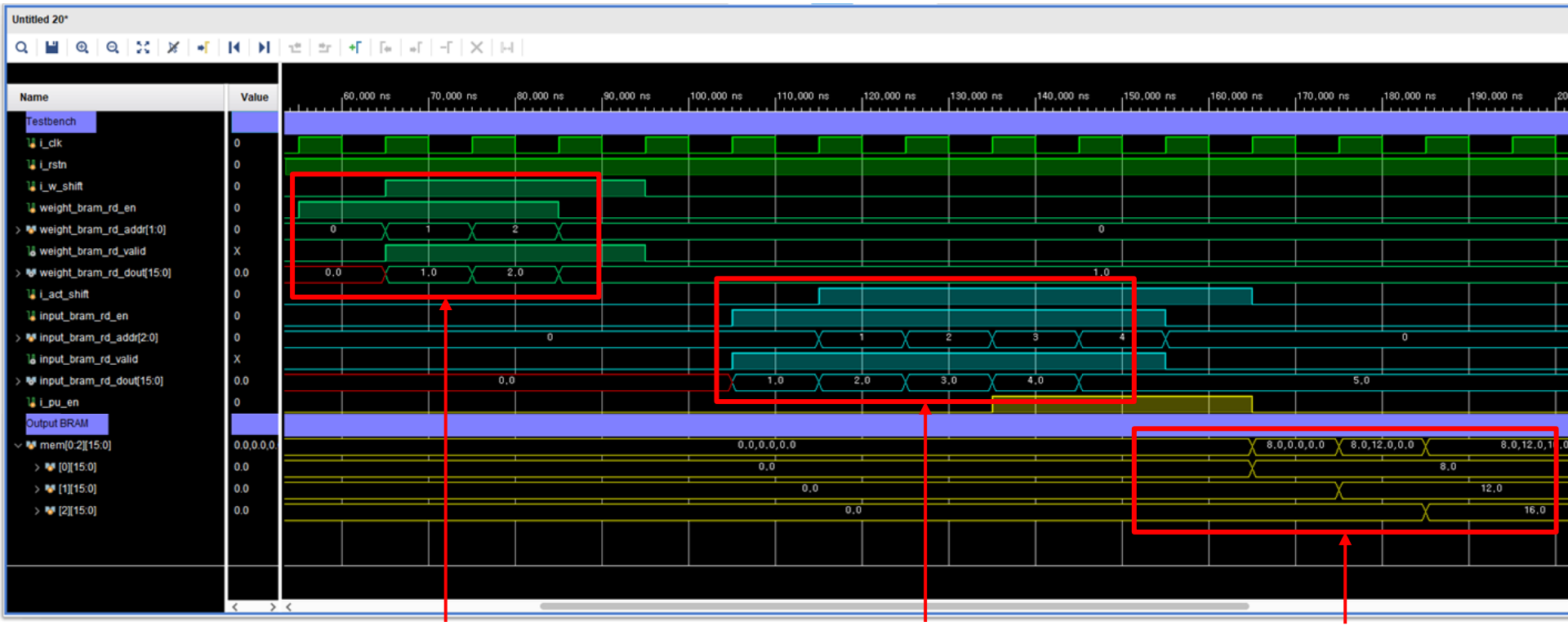
- **Don't forget the initialization BRAM (in Testbench).**

```
`timescale 1ns / 1ps

module simple_dual_port_bram #(
    parameter WIDTH      = 16,
    parameter DEPTH      = 1024,
    parameter ADDR_WIDTH = $clog2(DEPTH),
    parameter INIT_FILE  = ""
)(
    input  wire                 clk,
    input  wire                 wr_en,
    input  wire                 rd_en,
    input  wire [ADDR_WIDTH-1:0] wr_addr,
    input  wire [ADDR_WIDTH-1:0] rd_addr,
    input  wire [WIDTH-1:0]      wr_din,
    output reg                  rd_valid,
    output reg  [WIDTH-1:0]      rd_dout
);

    // BRAM
    (* ram_style = "block" *) reg [WIDTH-1:0] mem [0:DEPTH-1];
```

```
    generate
        if (INIT_FILE != "") begin : use_init_file
            initial begin
                $readmemh(INIT_FILE, mem);
            end
        end else begin : init_to_zero
            integer i;
            initial begin
                for (i = 0; i < DEPTH; i = i + 1)
                    mem[i] = {WIDTH{1'b0}};
            end
        end
    endgenerate

    always @(posedge clk) begin
        if (wr_en)
            mem[wr_addr] <= wr_din;

        if (rd_en) begin
            rd_dout <= mem[rd_addr]; // READ_FIRST
        end

        rd_valid <= rd_en;
    end

endmodule
```

Prob. 1 : Waveform (1)

Waveform



Weight Load

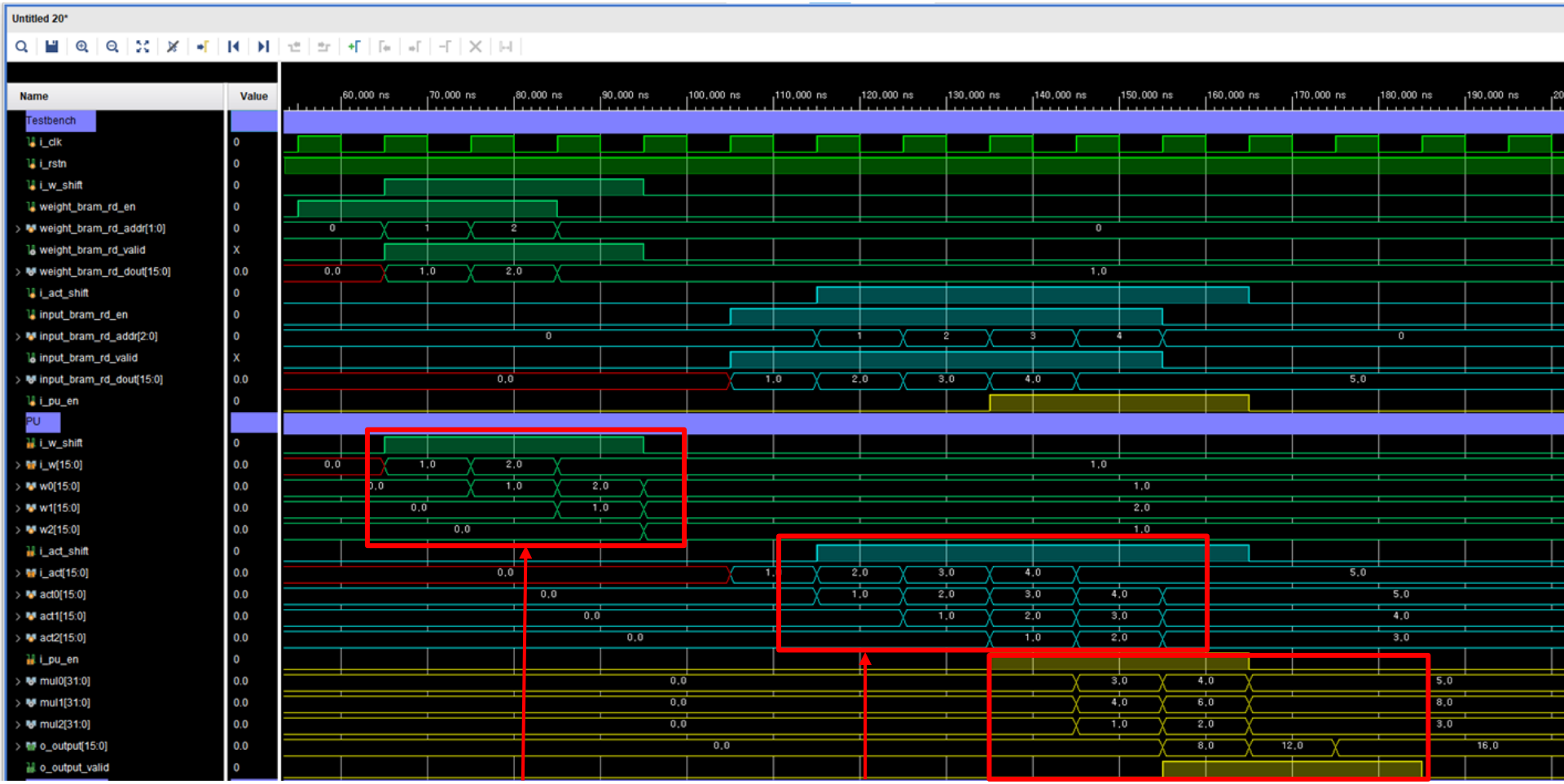
Input Load

Output Store

Prob. 1 : Waveform (2)

Waveform

- **After weight and input are shifted**, MAC operations are performed to generate the output.



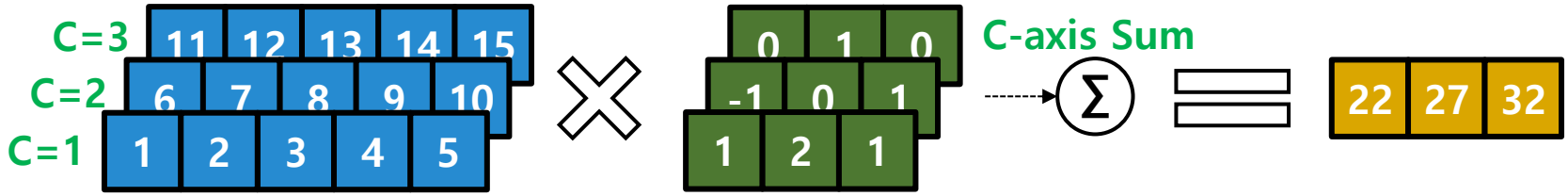
Contents

- Data Reuse
- Problem 1: Single Channel Conv1d (w/ 3 PEs)
- Problem 2: Multi Channel Conv1d (w/ 9 PEs)

Problem 2. Conv1d Objective

Multi-Channel Input Conv1d Operation Objective

- Generate **1×3 output from 3 input channels (C=3)** with **1×5 inputs** and **1×3 kernels.**
- Perform sliding-window-based dot product (MAC) **independently per channel.**
- Perform **channel-wise reduction (sum)** to produce the final output.

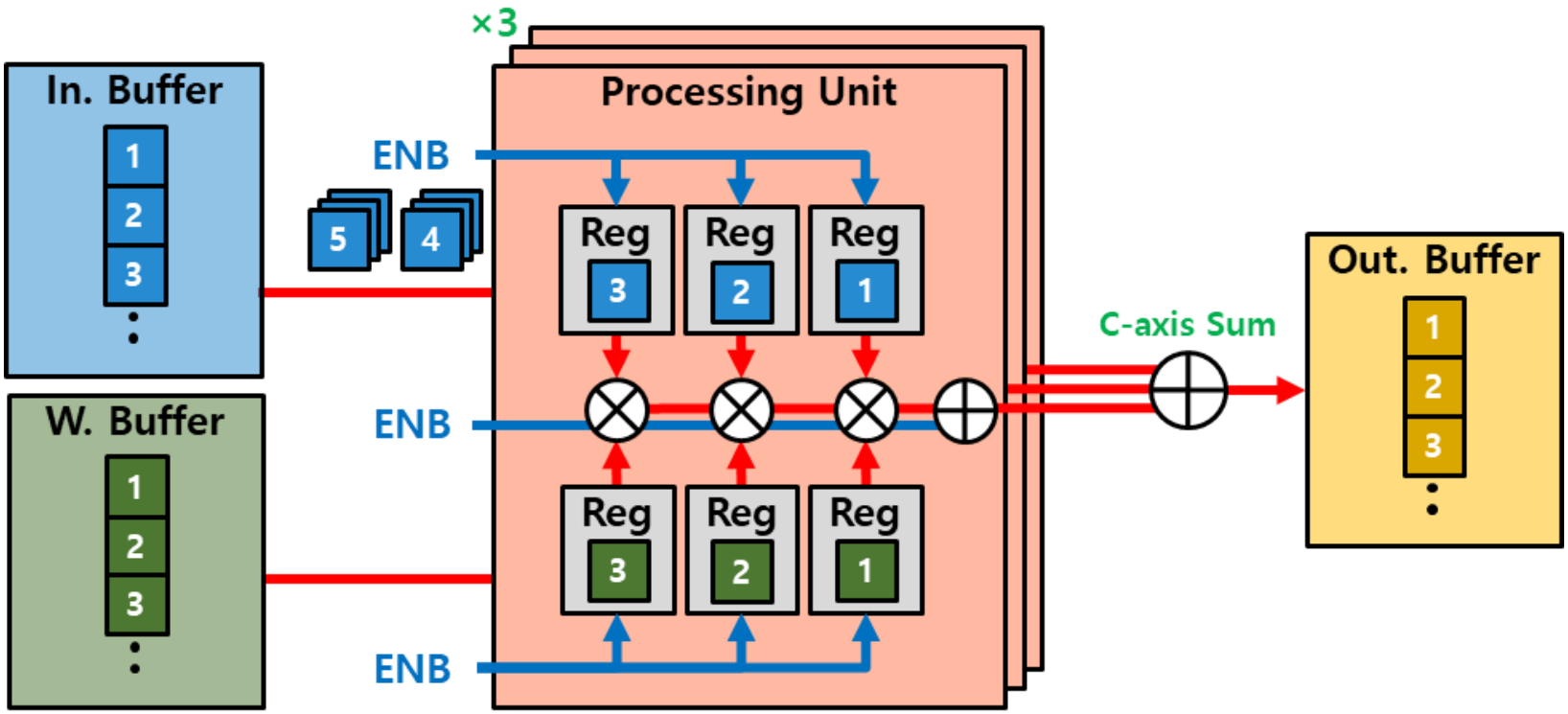


< Target Conv1d Operation >

Problem 2. HW Design Objective

Hardware Design Objective

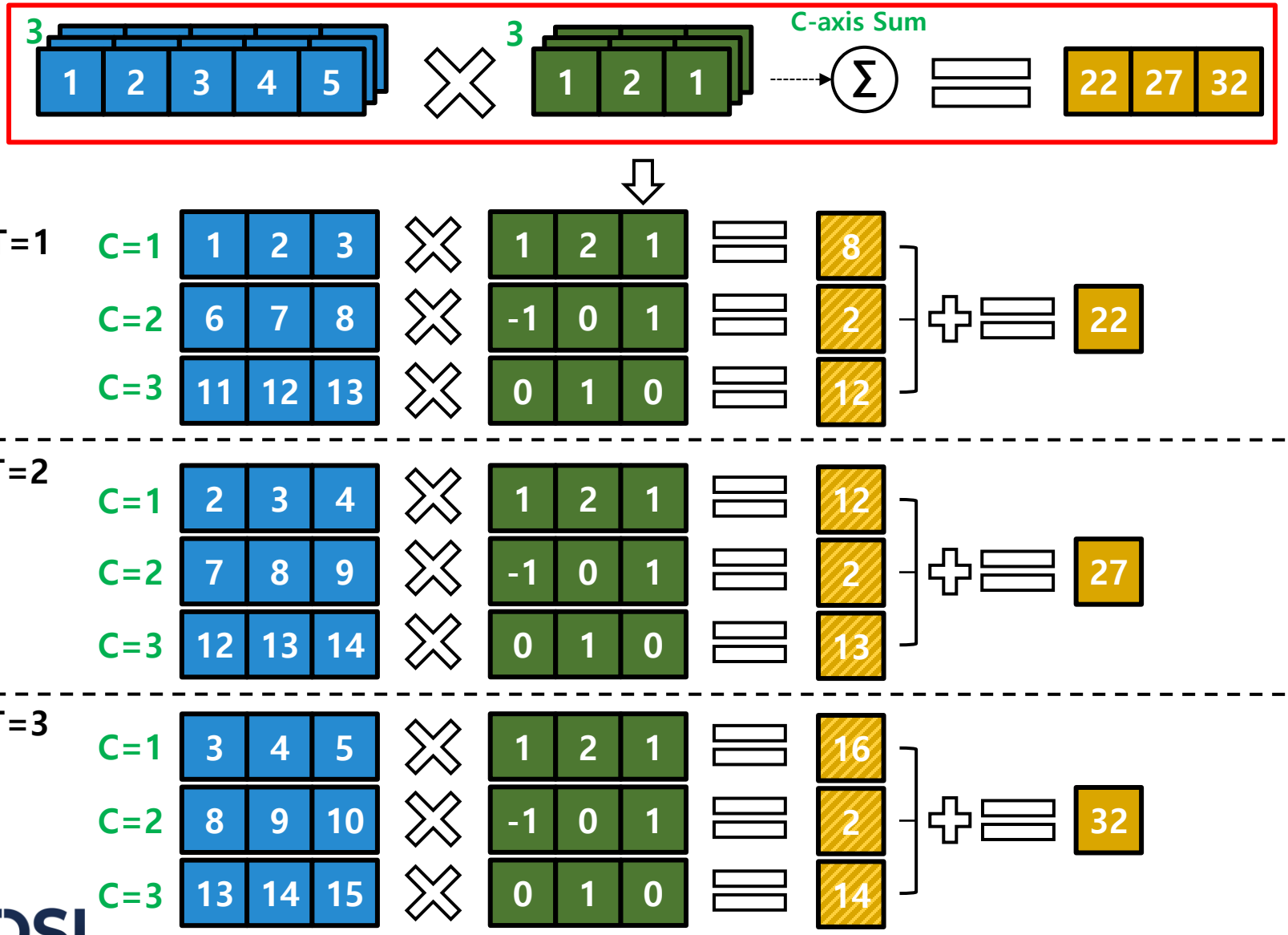
- Reuse the PU (in Problem 1) to perform **parallel MAC operations across channels.**
- Control data movement (shift) and computation using the **ENB signal.**
- **Accumulate channel-wise results** to generate the final output.



< Target Hardware Architecture >

Detailed Computation Flow

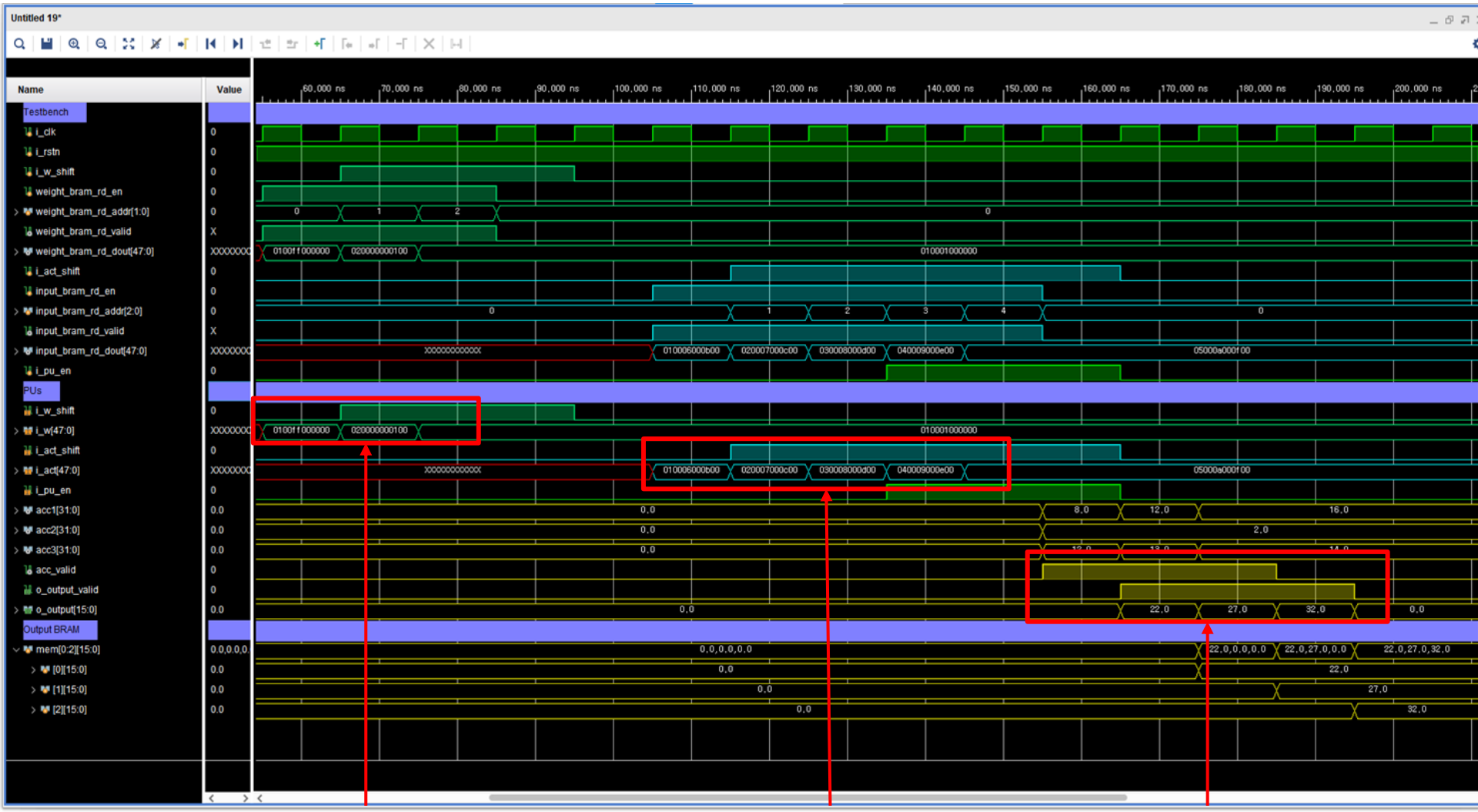
Target Conv1d: Detailed Computation Flow



IDSL
Intelligent Digital Systems Design Lab.
SEOULTECH

Prob. 2 : Waveform (1)

Waveform



Weight Load

Input Load

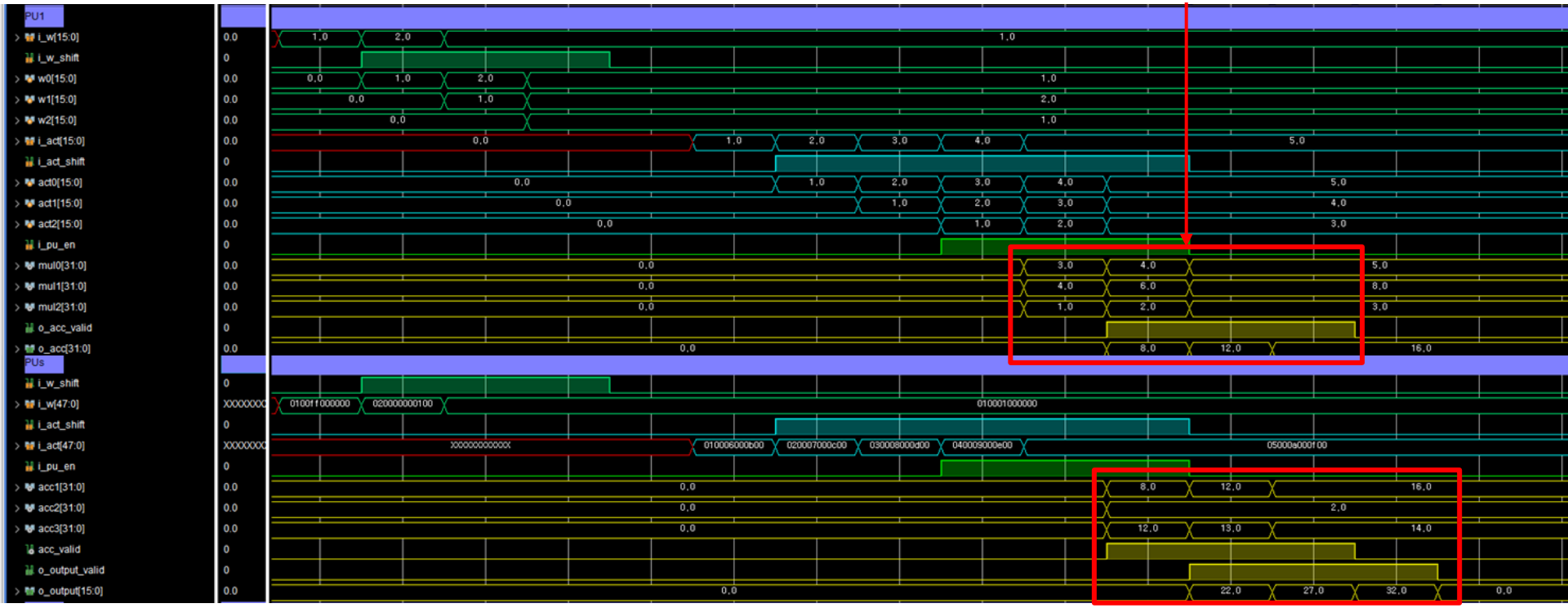
Output Generate

Prob. 2 : Waveform (2)

Waveform

- Channel-wise MAC is performed after loading weight and activation, followed by **reduction sum to produce the output.**

Dot-Product



C-axis Reduction Sum

Thank You!